

Day 174/365

Random Forest Regression in Python

Predicting a salary from a job level, step
by step.

Yesterday the idea. Today the code.

A random forest of trees, with random rows, averaged together. Now let's make one predict a number.

The task: predict an employee's **salary** from their **position level**.

Level	What it is
Level	The input (X): a number from 1 to 10
Salary	The answer we predict (y)

A tiny, classic practice dataset: just **10 rows**. Keep that in mind, it matters at the end.

Step 1: Separate inputs from the answer

```
# inputs: the Level column, kept 2-dimensional  
X = df.iloc[:, 1:2].values  
# the answer we want to predict  
y = df.iloc[:, 2].values
```

X is what the model looks at. **y** is what it must predict.

scikit-learn expects **X** as a **table** (rows and columns), even if it has only one column. That's why the slice is 1:2 and not just 1.

And no feature scaling is needed. Trees don't care about units, unlike the Ridge models in my car price series.

Step 2: Split into train and test

```
X_train, X_test, y_train, y_test =  
train_test_split(  
    X, y, test_size=0.2, random_state=42)
```

Same habit as my car price work: 80% to learn from, 20% kept hidden to check the model.

10

rows in total

8

to train on

2

to test on

Only **2 test rows**. Any score from that is shaky.
Remember this for slide 6.

Step 3: Build and train the forest

```
model = RandomForestRegressor(  
    n_estimators=100,  
    random_state=42,  
    oob_score=True)  
model.fit(X_train, y_train)
```

n_estimators=100: grow 100 trees.

random_state=42: same random draws every run, so results are repeatable.

oob_score=True: test each tree on the rows it never saw. That's the "left-out third" from yesterday's post, giving a free extra check.

Step 4: Predict and score

```
y_pred = model.predict(X_test)
mean_squared_error(y_test, y_pred)
r2_score(y_test, y_pred)
```

0.988

R²

616M

MSE

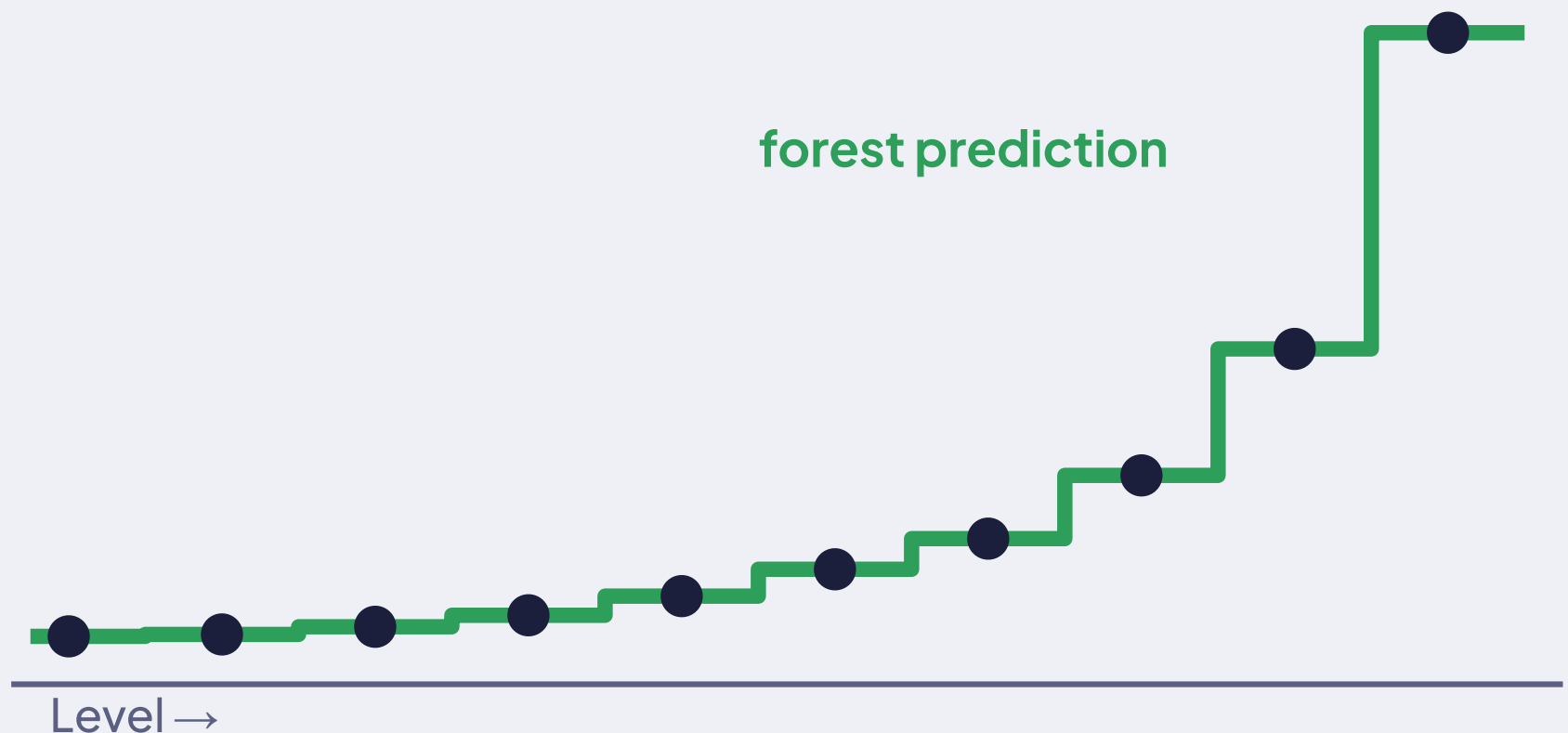
0.23

OOB score

R² near 1 means the model explains almost all the variation. **MSE** is in squared units, so take its square root: about 24,800 in salary terms.

R² of 0.99 looks great. But the OOB score is only 0.23. Both measure the same model. **Something doesn't add up.**

Why the forest draws a staircase



A tree answers with **one flat value per range**, like "Levels 6 to 7: 150K". Averaging 100 trees gives many small steps, not a smooth curve.

Compare my polynomial regression post: that gave a smooth bend. A forest gives **steps**.

Simplified sketch. A real forest's steps are finer.

Takeaways

Random forest regression is 5 lines of setup and one score to question.

`X (2D), y, split, RandomForestRegressor, fit, predict.`

No scaling needed. Predictions are the average of many trees, drawn as steps.

Tomorrow: R^2 says 0.99, OOB says 0.23. Which do we trust, and what's the catch with tiny datasets?